

palmsource™

PalmSource Seminar Series

Build Tomorrow ▸▸▸ Today

Presented By:

Dan Chernikoff
Sr. OS Manager
Bob Ebert
Sr. Applications Manager

palm™ POWERED

tapwave

a new mobile entertainment platform
to showcase your talents

overview

- company introduction
- target customer
- hardware overview
- software environment
- development tools and program
- API overview
- porting Palm applications to the Tapwave platform
- code examples
- q & a

company introduction

Founded in May 2001 by:

- **Peng Lim**, formerly Vice President of Worldwide Product Development for Palm, Inc.
- **Byron Connell**, formerly VP of Consumer Markets Group and acting VP of Marketing for Palm, Inc.

Strong leadership team including:

- **Marian Cauwet**, first VP of Engineering at Palm, Inc.
- **David Wenning**, formerly head of Channel Sales for the Americas, Palm, Inc.

PalmSource licensee since May 2001

Headquartered in Mountain View, CA

16 employees, multiple contractors, and extensive 3rd party resources

company introduction: our purpose

position

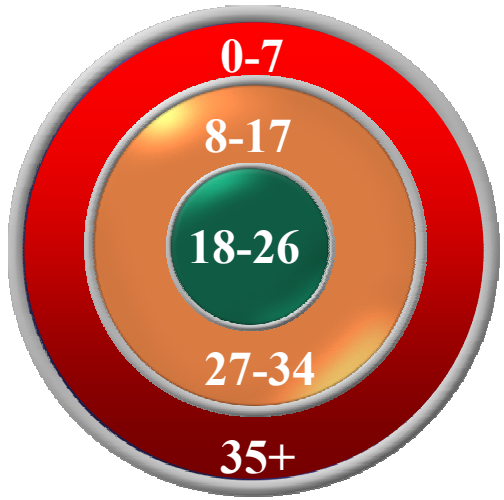
Tapwave gives
technology enthusiasts
(primarily ages 18-34)
the most
fun and function
in mobile
entertainment gear.

target customer: entertainment technology enthusiasts

PCs & Internet Use

- **69%** of teens use a PC at home, school, or work (U.S.)
- **73%** of 16-22 year olds will be online by 2004 (U.S.)

Target demographic



Video Games – "The Nintendo Generation"

- Playing Video Games is ranked as their #1 leisure activity
- **70%** of children live in homes with a video game player (U.S.)
- **57%** of males age 18-24 own gaming systems (U.S.)

Digital Imaging – Creative Expression

- **47%** say they would like to own a digital camera.
- Males ages 18-24 are adopting digital cameras 4 times faster than the avg. consumer.

Music – "The MTV Generation"

- **47%** of teens have portable CD players.
- Downloaded digital music projected to grow over 170% per year to $1.1B (2004).

the customer perspective



Comments on Game Boy Advance

- "The worst thing about it is that you can't see it"
- "Please make it rechargeable!"
- "Needs at least 2 more buttons for gaming"
- "Make it more ergonomic for your hands"
- "Speakers should be on both sides"
- "Navigation should be more sensitive"

Comments on Tapwave Concept

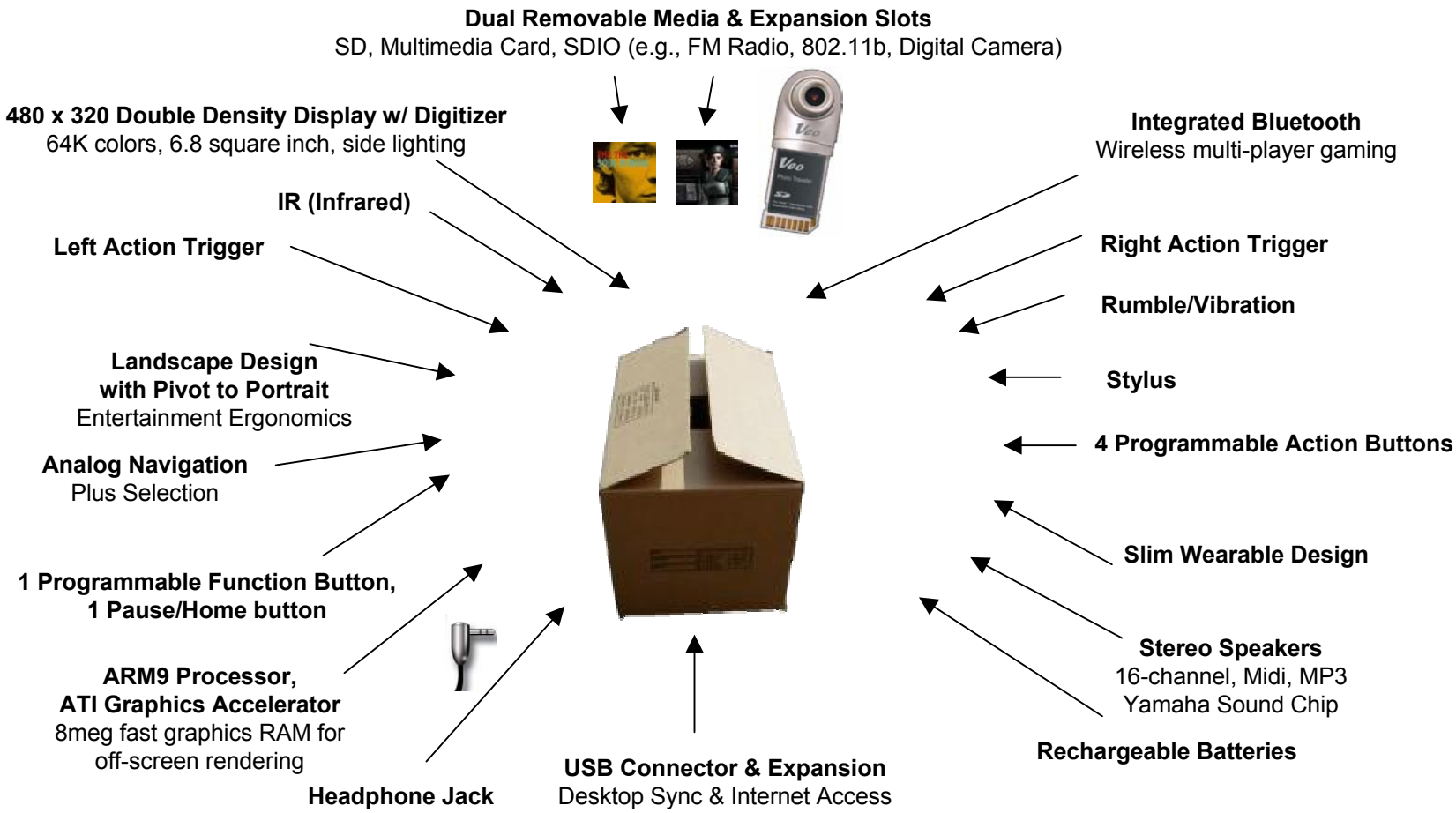
- "Wireless games would be really cool"
- "MP3 would be cool…a must have".
- "Would like support for more than 4 players"
- "E-book would be a good thing to have"
- "Would like a built in alarm or reminder"
- "A graphing calculator would be good for math"

you've got a vision.
we've got a platform for that vision.

Tapwave's products deliver:

- Best-in-class technologies
- Player-centric design
- Advanced gaming and entertainment features
- Flexibility for your application development
- Access to a new customer base
- New channels of distribution

hardware overview



software environment

PalmOS 5.2.1

- Large RAM sizes (>16 MB)
- Graffiti 2

Tapwave Modifications

- Native ARM applications (ARMlets with debugging and OS APIs)
- Improved HotSync – fixes 64K limitation
- New input event queue model
- PINS and status bar support
- Large 480 x 320 double density screen, landscape and portrait modes
- Forward binary compatibility with PalmOS 6.0

Bluetooth Motown Café APIs

- Binary compatiblity with Palm Bluetooth applications

development tools

PalmSim Simulator

- Supports Landscape and Portrait Screen Modes
- Special Input hardware via GamePad controller
- Large Form layout
- Requires MS Visual C

Native Arm Applications

- Metrowerks CodeWarrior 9.2
- Tapwave CodeWarrior Update Patch

development tools (continued)

Sample Code – “NASCAR” Game Application demonstrates

- Using the Tapwave API to use the full screen size.
- Using the Tapwave Graphics API (TwGfx) for drawing.
- Using the Tapwave Input API (TwInput) to scroll.
- Writing a game as an extended ARMlet using CodeWarrior 9.
- Interacting with Palm Operating System safely (low battery warnings, alarms, exiting).

developer program

1. Idea Generation

Easy to get started:
- Low barriers to entry
- Solution inspirations
- Quick start guide
- Reference materials



2. Product Development

Tools for rapid development:
- Game engine
- Development devices
- SDK, APIs
- Documentation



3. Media Card Production Services

Full service media duplication, labeling, packaging and distribution



4. Promotion

Promote Developer Solutions to TapWave Customers:
- Co-marketing opportunities
- Trade Shows
- Website features
- Solution showcases
- Customer Newsletter



5. Distribution

Customer & Channel Access:
- In Store
- Joint merchandising
- Low cost internet distribution



The TapWave Developer Program:
Designed to accelerate development, improve access to the channel and target customer, and increase the likelihood of market success.

API overview

System-level APIs

Graphics APIs

Sound APIs

Gaming Support

Music and Photo Support Libraries

APIs: system-level

New Event Queue Model

- Simultaneous Button Presses
- Analog and/or D-Pad Navigator Input

Screen APIs

- PINS from PalmOS for Graffiti, Status Bar
- Tapwave addition for Orientation (Landscape/Portrait)

Virtual Device APIs

- Rumbler
- Audio Amplifier

Bluetooth – Motown Café APIs

APIs – graphics and sound

TwGraphics (Low-Level)

- A lightweight fast 2D graphics API
- High performance - a very thin layer around the hardware
- Lightweight - it's easy to use and program
- 2D - provides a common set of 2D rendering operations
- Portable - runs on the device as well as the simulator

TwSound (Low-Level)

- Thin layer around sound chip

PalmOS Graphics (Win*) and Sound (Snd*) APIs

Fathammer Libraries

- Device and Platform Independence
- X-Forge Core
- Gaming Engine

APIs: graphics and sound
Fathammer

The X-Forge™ 3D Game Engine is a complete cross-platform mobile game development toolkit that includes:

- Advanced 3D graphics rendering and multi-channel audio
- Collisions and physics
- Character / entity systems and artificial intelligence
- Breathtaking particles, flares and other eye-candy effects
- Multi-player engine for Bluetooth / Wireless LAN gaming
- Special optimizations for different game genres
- Content development and converting tools

Based on established PC and console industry game engine architectures (C++ based, APIs similar to DirectX and OpenGL)

APIs: gaming support

High Score Support w/Conduit & Website

- Simple interface for storing high scores, backup included
- Dual opt-in for web server support
  - Game must request server sharing
  - User must enable server sharing
- Can extend to retrieving high scores from server
- Basis for wider gaming (tournaments, clubs, ladders)

Digital Rights Management

- Verifiable user account IDs using digital signatures
- Based on unchangeable Device, SD Card, or ROM Card IDs
- OS utilities to verify apps that are locked to card, device, or user
- Server utilities to sign applications to card, device, or user
- (Not for data/media rights management)

APIs: support libraries

Playlist Support

- Enumerate available songs or photos
- Filter by storage location
- Sort by custom, alphabetical, shuffle
- Query for song- or photo-specific details (track name, photo **size)**

MP3 Decoder Library

- Uses PalmOS sound manager, working on background playback
- Higher level APIs (play/pause/seek, based on canonical file name)

JPEG Decoder Library

- Uncompress to a given width/height.
- Based on canonical file name

Extensible to other music or graphic formats

- Future: Ogg Vorbis? PNG? GIF?

porting existing PalmOS apps to Tapwave

Intention is 100% compatibility for “well behaved” PalmOS apps

Watch out for:

- Direct screen access
  - 16-bpp double density screen, hardware accelerated
  - base address & origin changes with centering, left-handed mode
  - RowBytes changes with orientation
- Assumptions about input (Palm’s 5-way nav, app/scroll buttons)

Things to Add:

- Enable full screen, landscape preferred, consider centering
- Use analog joystick, triggers, action buttons
- Follow play/pause/exit guidelines

Plan of Attack

- Test your app under our PalmSim w/debug prototype ROM
- Talk to us about what you find

code examples

Handling Screen Size Changes: PINS spec

- During **frmLoadEvent/frmOpenEvent**: (initialize your app)
  - Screen is initially landscape (normally), with graffiti and status open.
  - **PINSetInputAreaState**(pinInputAreaShow, -Hide, -User)
  - **StatShow**() or **StatHide**() - standard apps should leave status open
  - **TwScreenSetOrientation**(twOrientationPortrait, -Landscape)
  - **WinGetWindowBounds**(**WinGetDisplayWindow**(), &bounds)
- During **winDisplayChangedEvent**: (graffiti, status, orientation change)
  - Compute deltaX and deltaY: compare form bounds with display bounds
  - Update your form size: **WinSetWindowBounds**(**FrmGetWindowHandle**(),…)
  - Change form object positions: **FrmGetObjectBounds, FrmSetObjectBounds**
- During **winEnterEvent**: (obscuring dialog just closed)
  - Games should pause
  - If necessary, set up graffiti, status, and orientation

code examples

User Input (Analog Joystick, Game Buttons)

- **TwInputOpen**(&queueP, “twinput”, “r”), **TwInputClose**(queueP)
- **TwInputSetFormat**(queueP, &format, formatSize * sizeof(Int32))
  - Format is just an array of **Int32**s, each is a requested data type
  - Formats specify what data you want to read, subset of:
    - Pen state: X, Y, Up or Down. (Only over app area)
    - Joystick state: cartesian X, Y or polar R, Theta, CircX, CircY
    - Joystick press (“Select” button): Up or Down
    - Virtual 4-way or 8-way rocker: Up/Down/Left/Right + Center
    - Action Buttons: A, B, C, D; Up or Down
    - Triggers: Left, Right; Up or Down
    - Function button: Up or Down
- **TwInputPoll**(queueP, &myEvent, sizeof(MyEventType)
  - **MyEventType** is a struct of **Int32**s that matches the format
  - Simultaneously gives current state of all controls

code examples

User Input (Fast Queues)

- **TwInputActivate**(queueP) to start queueing (not needed if you poll)
- **TwInputDeactivate**(queueP) to stop queueing
- **TwInputRead**(queueP, *myEvent, sizeof(MyEventType))
  - Reads data from the queue
- **err = TwInputPeek**(queueP, *myEvent, sizeof(MyEventType))
  - **err = EAGAIN** if there are no events, myEvent is optional
  - does not dequeue event
- **TwInputSetPeriod**(queueP, milliseconds) to control poll frequency
  - Not needed if you only want data when a control changes state
- **TwInputSetCapacity**(queueP, capacity) to set queue size
  - Controls how many events to queue
  - Any transition is a whole new event
  - Default is 0 (only current state), max is 16.

code examples

TwGraphics Example

- General APIs
  - **TwFgxOpen**() and **TwGfxClose**() to get access to the graphics library
  - **TwGfxWaitForVBlank**(gfxType) to wait for VBLANK to begin
- Surface Manupulation APIs
  - **TwGfxAllocSurface**(gfxType, &result, gfxSurfaceDescription) to allocate a new surface, for use by rendering calls. **TwGfxFreeSurface**() to free.
  - **TwGfxGetSurfaceInfo**() to get info about a surface
  - **TwGfxGetDisplaySurface**() to get handle to surface representing the display
  - **TwGfxSetClip**(), **TwGfxGetClip**() to set and get clipping rectangle on surface
  - **TwGfxLockSurface**() and **TwGfxUnlockSurface**() to lock surface in memory
  - **TwGfxWriteSurface**() writes buffer to display
  - **TwGfxReadSurface**() makes a copy of the surface display memory

code examples

TwGraphics Example (cont)

- Rendering Calls
  - **TwGfxBitBlt** (destSurface, destPoint, srcSurface, srcRect)
  - **TwGfxTransformBlt** (destSurface, destPoint, srcSurface, srcRect, rotationFlags, mirrorFlags) to mirror and rotate in 90-degree increments
  - **TwGfxDrawPoints**(destSurface, &points, numPoints, color) to draw points
  - **TwGfxDrawColorPoints**(destSurface, &points, numPoints, &colors)
  - **TwGfxDrawLines**(destSurface, &points, numPoints, color)
  - **TwGfxDrawLineSegments**(destSurface, &points, numPoints, color)
  - **TwGfxDrawRect**(destSurface, rect, color)
  - **TwGfxFillRect**(destSurface, rect, color)
  - **TwGfxDrawSpans**(destSurface, &spans, numSpans, color)
  - **TwGfxDrawBitmap**(destSurface, destPoint, bitmap, transparentBG)

code examples

Sound Example

- UInt16 TwSndGetVolume()
  TwSndSetVolume(UInt16 newVolume)
  - Get and set the application volume, 0..64,
  - Saved prefs, controls hardware amplifier
  - Old “game/app” sound pref is permanently full volume
    - (but APIs to change it are patched)
- Boolean TwSndGetMute(UInt32 *muteUntilP)
  TwSndSetMute(Boolean mute, UInt32 muteUntil)
  - Get and set the mute state, device-wide, alarm unmutes via Sound panel
- UInt16 TwSndGetBassBoost()
  TwSndSetBassBoost(UInt16 boostLevel)
  - Get and set bass boost level, 0..64

code examples

Fathammer Game Example

See the Fathammer website for documentation and examples

http://www.fathammer.com

code examples

Rumbler example

- **TwDeviceOpen**(&deviceHandle, “rumbler0”, “rw”)
- **TwDeviceClose**(deviceHandle)
- **TwDeviceControl**(deviceHandle, cmd, &buffer, length)
  - Cmd = **strt**, **stop**, **play**, **rept**, **fast**, **medi**, **slow**
  - Buffer = array of UInt8 pairs: speed and duration, for **play** and **rept**
  - Length = number of array elements in bytes
- Can turn vibrate on “forever” using **strt/fast**, **medi**, **slow**
- Turn vibrate off no matter what using **stop**
- Can play an effect from a library using **play** or **rept**
  - Speed is 0-255, 0 = off
  - Duration is in 1/100ths of a second
  - Example fast, slow, fast:
    [‘\255’, ’\10’, ‘\0’, ‘\10’, ‘\127’, ’\10’, ‘\0’, ‘\10’, ‘\255’, ’\10’, ‘\0’, ‘\10’]

code examples

High Score example

- **TwHighScoreRegister**(myCreatorID, 0 /* score type */,
  10 /* local scores */, 0 /* server scores */,
  false /* write scores to server */, false /*get server scores */,
  “MyCoolGame")
  - Only needs to be called once, but no harm in calling it every launch
- **TwHighScoreUnregister**(myCreatorID, 0)
  - Deletes all high score data, only called during “delete me” cleanup
- **TwHighScoreReport**(myCreatorID, 0, &highScoreData)
  - Data contains UInt32 score, DateTime, and a game-specific string
- **TwHighScoreGetSummary**(myCreatorID, 0)
  - Returns number of scores, highest/lowest, server & local, sync time
- **TwHighScoreGetDetails**(myCreatorID, 0, Local/Server, index)
  - Returns details about an individual score entry

code examples

Digital Rights Management example

- **Currently Under Development**
- **Check Back Later**

q & a

Meet the Engineering Team

Software: Hong Zhang, Manas Deb
Roger Flores, Chris Raff, Kipp Hickman, Ben Bostwick
Bob Ebert, Dan Chernikoff

Hardware: Allen Lee, Roy Riccomini

Quality Assurance: Yannick Bertolus

Product Marketing: Scott Lincke, Tim Twerdahl, Dawn Hannah

Dev Support & Programs : Joyce Morrell, Mitzi McGilvray, Ryan Robertson





go. do. play.